claude-windows / df5d5fb2-534a-447a-8181-b222f94655f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\df5d5fb2-534a-447a-8181-b222f94655f3\subagents\workflows\wf_f6b96576-946\agent-a9956fb0da828826e.jsonl`
- SHA-256: `3b01b196314b16702fe9ebd04df7e6f7add58600f64a0ee78b6cd968312323e4`
- Source modified: `2026-07-06T18:07:23+00:00`
- Imported at: `2026-07-08T15:59:36+00:00`
- project: `wf_f6b96576-946`
- session_id: `df5d5fb2-534a-447a-8181-b222f94655f3`

Transcript

1. user (2026-07-06T18:06:12.056Z)

```
You are the SYNTHESIS lead for the wf294 F1/F2/F3 completeness review. Below are findings that
SURVIVED adversarial verification. Dedupe, drop any fine-on-reflection, rank most-severe-first,
and
give an overall verdict on whether this is safe to open+merge a PR as-is or needs changes first.
Mark each finding must_fix_before_merge true/false. Do not invent findings not in the list.


REPO ROOT: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
FULL DIFF UNDER REVIEW (read first): C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/df5d5fb2-534a-447a-8181-b222f94655f3/scratchpad/wf294_full.diff

BACKGROUND: wf294 = the Cloud Run rally-worker recorded status="success" / 0 segments even when
the WASB detector TIMED OUT and produced no trajectory ? a detector failure indistinguishable
from
a legit 0-rally video. An earlier change (already reviewed CLEAN) fixed the two core detector
paths
(healthcheck-not-ready, no-trajectory-abort) to return backend.results.Failure, and made
worker.cmd_infer's _fail() write+push a status="failed" report.json + done-marker.

THIS REVIEW covers the FOLLOW-UP completeness work that closed the same silent-failure CLASS on
adjacent paths (a prior review confirmed these were the last gaps). Three sub-changes:

F1 ? backend/pipeline/segmenters/trajectory_hybrid.py process_video: the pre-detection
config/env/
   input abort paths now return Failure instead of bare []: unknown sport (KeyError), missing
API
   key (skip_ai=False), unknown provider (KeyError), invalid/<=0 video duration, and
_resolve_runner
   NotImplementedError (detector_impl=native on a family with no native runner). A genuine empty
   match (trajectory produced but zero candidate windows) still returns bare [].

F2 ? backend/pipeline/segmenters/yolo_hybrid.py (the model-default segmenter): same treatment ?
   _resolve_provider now returns Failure (not None) on bad sport / missing key / unknown
provider;
   process_video returns Failure on invalid duration; return annotation widened; imports
Failure.

F3 ? the AI-handoff "all windows errored" path in BOTH trajectory_hybrid.py (inline Phase 3) and
   yolo_hybrid.py (_run_ai_handoff): the per-window handoff fn now returns (valid_segments,
errored)
   where errored=True iff the window yielded NO verdict (clip extraction failed OR provider
   metrics['error']). If candidate windows > 0 AND every handed-off window errored,
process_video
```

   returns a Failure(is_recoverable=True). If ANY window ran (even returning no rallies), it
stays a
   Success (bare list). skip_ai (fully-local) has no provider and never reaches this path.

CONTRACT: every caller (worker.cmd_infer:326, main.py:1557, orchestration.py:806) branches on
isinstance(result, Failure). worker._fail writes status="failed" report.json + marker.
trajectory_hybrid is NOT mypy-quarantined (must stay mypy-clean); yolo_hybrid + worker.__main__
ARE
quarantined (mypy.ini ignore_errors). CI: ruff 0.15.16, mypy 2.1.0 (mypy backend training),
run_all_tests.py, coverage floor 70%. Current state: ruff clean, mypy clean, 1740 passed / 3
skipped, coverage 85.85%. So do NOT speculate that "tests might fail" ? cite concrete code
defects.

Report ONLY real defects with exact file:line: a wrong/edge behavior (off-by-one, an all-errored
run that stays "success", or a partial/legit-empty run wrongly failed), a broken contract, a
thread-safety/ordering bug, a regression vs the prior behavior, a convention violation, or a
test
that asserts the wrong thing / has a gap. If you cannot ground a concern in the actual code, DO
NOT
report it.

SURVIVING FINDINGS (JSON):

```
[
  {
    "dimension": "completeness-regression",
    "severity": "medium",
    "verdict": "CONFIRMED",
    "file": "tests/test_trajectory_hybrid_equivalence.py",
    "line": 157,
    "summary": "The new test_partial_handoff_error_is_a_success_not_a_failure binds
provider.analyze_video results to windows by call-ARRIVAL order (side_effect list) while the two
windows are handed off CONCURRENTLY (ai_max_workers default = 5), so which window gets the rally
vs the error is non-deterministic and the coordinate-offset assertion can spuriously fail.",
    "detail": "process_video runs the handoff in a
ThreadPoolExecutor(max_workers=ai_max_workers); the config is {\"indexing\": {}} so
ai_max_workers = 5 (backend/config/models.py:254). trajectory_hybrid.py lines 468-477 submit ALL
handoff windows up front, so with 2 windows the executor spawns 2 worker threads that both call
provider.analyze_video concurrently. provider.analyze_video.side_effect is a list ([rally], then
[error]) and a MagicMock consumes side_effect entries in the order calls ARRIVE, not in window-
submission order. The test expects window 0 (w_start = max(0, 1.0-2.0)=0.0) to receive the rally
so the offset segment is 0.5+0.0=0.5. But if window 1's thread (w_start = max(0,10.0-2.0)=8.0)
reaches analyze_video first, it receives the rally and the offset segment becomes 0.5+8.0=8.5.
The result aggregation itself is order-safe (window_results keeps per-future tuples), so the run
is still correctly a Success ? but the assertion `res[0][\"start_time\"] == 0.5` depends on
thread scheduling and can flake. handoff_padding default = 2.0 (models.py:253). No other multi-
window side_effect test exists in the suite, so there is no precedent proving the executor
happens to serialize.",
    "failure_scenario": "Under load/CI the second thread wins the race to provider.analyze_video
and consumes the [rally] side_effect entry; the surviving segment is offset by window 1's
w_start (8.0), so res[0][\"start_time\"] == 8.5, the assertion `res[0][\"start_time\"] == 0.5`
fails, and the test flakes intermittently. (len(res)==1 still holds, so it looks like a
mysterious value mismatch.)",
    "suggested_fix": "Make the window->result mapping deterministic instead of arrival-order:
either force serialization by running with ai_max_workers=1 (e.g. cls(db, {\"indexing\":
{\"ai_max_workers\": 1}})), or replace the ordered side_effect list with a side_effect FUNCTION
that keys off the clip path / window (e.g. returns the rally only for the handoff_..._0.mp4 clip
and the error otherwise), so the rally is always bound to window 0 regardless of thread
scheduling.",
    "verifier_reasoning": "Verified every load-bearing claim against the code as written:\n\n1.
Concurrency: trajectory_hybrid.py:468-477 submits ALL handoff windows up front into
ThreadPoolExecutor(max_workers=ai_max_workers). With 2 windows and ai_max_workers=5, both worker
threads run concurrently. ai_max_workers is read from idx_cfg (line 270) whose default is 5
(models.py:254); the test config {\"indexing\": {}} (test line 178) supplies no override, so 5
is in effect. Nothing serializes the two threads between submission and the
provider.analyze_video call ? extract_video_chunk is mocked to return True instantly (test lines
```

172-173) and get_prompt/analyze_video are MagicMocks, so both threads reach analyze_video near-simultaneously.\n\n2. Arrival-order binding: provider.analyze_video.side_effect is a 2-element list (test lines 158-161: [rally] then [error]). A MagicMock consumes side_effect entries in the order calls ARRIVE, not by window index wi. The offset applied to a returned segment is w_start (line 452: seg[\"start_time\"] = float(...) + w_start).\n\n3. Offsets: w_start = max(0, window[\"start\"] - handoff_padding) (line 426), handoff_padding default 2.0 (models.py:253). Window 0 (_window(1.0,4.0)) -> max(0, 1.0-2.0)=0.0; window 1 (_window(10.0,13.0)) -> max(0, 10.0-2.0)=8.0. So the rally lands at 0.5+0.0=0.5 if window 0's thread calls analyze_video first, or 0.5+8.0=8.5 if window 1's thread wins the race.\n\n4. The assertion (test line 181) is `res[0][\"start_time\"] == 0.5`, which holds only when window 0 arrives first. The aggregation is order-safe (window_results is a per-future list, lines 477-480), so len(res)==1 always holds and the run stays a Success ? confirming the finding's precise characterization that this is an intermittent VALUE mismatch, not a structural failure.\n\nNo guard rebuts this: there is no lock, no submission-order enforcement, and no keying of side_effect by clip path/wi. The failure scenario is concretely reproducible under adverse thread scheduling. This is a genuine flaky-test regression introduced by the new test. Medium severity is right: impact is intermittent CI flakiness in a newly added test, not a production defect."
  },
  {
    "dimension": "completeness-regression",
    "severity": "medium",
    "verdict": "CONFIRMED",
    "file": "tests/test_yolo_hybrid.py",
    "line": 174,
    "summary": "test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure has the identical race: an ordered analyze_video side_effect list bound by call-arrival order while _run_ai_handoff hands off the two windows concurrently (ai_max_workers default 5), so res[0][\"start_time\"] can be 8.5 instead of 0.5.",
    "detail": "yolo_hybrid._run_ai_handoff (yolo_hybrid.py:399-407) submits all candidate windows to ThreadPoolExecutor(max_workers=ai_max_workers); config {\"indexing\": {\"use_gpu\": False}} leaves ai_max_workers at its default 5 (models.py:254). two_windows = [{start:1.0,end:4.0}, {start:10.0,end:13.0}] with handoff_padding default 2.0 give w_start 0.0 and 8.0 respectively. mock_provider.analyze_video.side_effect = [([rally], {}), ([], {error})] is consumed in call-arrival order; if window 1's thread calls first it receives the rally and the confirmed segment is offset to 0.5+8.0=8.5, breaking `res[0][\"start_time\"] == 0.5`. The Success-vs-Failure verdict is still correct (aggregation is order-safe); only the value assertion is racy.",
    "failure_scenario": "In CI the window-1 thread reaches mock_provider.analyze_video before the window-0 thread, consumes the rally side_effect entry, and the single surviving segment lands at start_time 8.5; the assertion `res[0][\"start_time\"] == 0.5` fails and the test flakes.",
    "suggested_fix": "Same remedy as the trajectory twin: run this scenario with ai_max_workers=1 (pass {\"indexing\": {\"use_gpu\": False, \"ai_max_workers\": 1}}), or use a side_effect FUNCTION that returns the rally only for the window-0 handoff clip (match on the w_path / wi) so the rally is deterministically bound to window 0 irrespective of thread arrival order.",
    "verifier_reasoning": "Verified against real code. In test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure (tests/test_yolo_hybrid.py:164-193), the config is {\"indexing\": {\"use_gpu\": False}} which leaves ai_max_workers at its default 5 (backend/config/models.py:254) and ai_handoff_padding at 2.0 (models.py:253). _run_ai_handoff submits ALL candidate windows to ThreadPoolExecutor(max_workers=ai_max_workers) (yolo_hybrid.py:399-407), so the two windows [{start:1.0,end:4.0},{start:10.0,end:13.0}] run concurrently. Each thread calls provider.analyze_video (yolo_hybrid.py:363) which is mocked with a plain ordered list side_effect (test line 174-177): [([rally@0.5],{}), ([],{error})]. MagicMock consumes side_effect entries in call-ARRIVAL order, not submission index, so if window-1's thread reaches the call first it receives the rally. w_start for window 0 = max(0,1.0-2.0)=0.0 and for window 1 = max(0,10.0-2.0)=8.0; the coordinate mapping at line 382 adds w_start, so a rally bound to window 1 lands at 0.5+8.0=8.5, breaking the assertion `res[0][\"start_time\"] == 0.5` (test line 193). The Success-vs-Failure verdict (line 192) and len(res)==1 (only one window errors, handoff_failed=1 < 2, so line 423 does not fire) are order-safe; only the value assertion is racy. All claimed line numbers, defaults, and behaviors match the code as written. Severity medium is appropriate: a genuine flaky-test race that only surfaces the wrong value, not a production defect."
  },

```
  {
    "dimension": "convention-tests",
    "severity": "low",
    "verdict": "CONFIRMED",
    "file": "backend/pipeline/segmenters/trajectory_hybrid.py",
    "line": 225,
    "summary": "The missing-API-key Failure sets is_recoverable=True with a comment claiming it
'mirrors gemini', but the gemini segmenter (the cited baseline) returns is_recoverable=False for
the identical missing-key case.",
    "detail": "trajectory_hybrid.py:225 returns `Failure(msg, is_recoverable=True)` for the
missing-API-key path, and its comment at line 223 says 'mirrors gemini'. yolo_hybrid.py:98 does
the same (`Failure(msg, is_recoverable=True)`, docstring at line 71 says 'mirrors the
gemini/trajectory segmenters'). But the actual gemini baseline at
backend/pipeline/segmenters/gemini.py:116 returns `Failure(msg)` for the SAME missing-key case,
i.e. the default is_recoverable=False. So the same failure class (missing key) is now classified
inconsistently across the three segmenters, and the 'mirrors gemini' comment is factually
contradicted by gemini's code. This is not a runtime bug today: a repo-wide grep for
`.is_recoverable` finds ZERO backend consumers (only tests read it), so no caller branches on
it. But it is a latent inconsistency ? a future consumer that acts on is_recoverable (e.g.
retry-vs-fail routing) would treat gemini's missing-key as non-recoverable and trajectory/yolo's
as recoverable, for the identical condition ? plus a stale/incorrect comment. The other
converted paths (unknown sport/provider bare `Failure(str(e))`, invalid-duration bare
`Failure(...)`) DO match gemini's default-False convention; only the missing-key line
diverges.",
    "failure_scenario": "A future change adds retry logic keyed on Failure.is_recoverable. A
missing GEMINI_API_KEY produces a non-recoverable Failure via the gemini segmenter but a
recoverable (retried) Failure via trajectory_hybrid/yolo_hybrid ? divergent handling of an
identical misconfiguration, with the 'mirrors gemini' comment actively misleading the person
making that change.",
    "suggested_fix": "Pick one convention for the missing-key class and apply it to all three
segmenters. Either drop `is_recoverable=True` at trajectory_hybrid.py:225 and yolo_hybrid.py:98
to match gemini.py:116, or (preferably, since a missing key IS operator-recoverable) set
is_recoverable=True in gemini.py:116 too. Then fix the 'mirrors gemini' comments so they match
reality.",
    "verifier_reasoning": "All factual claims verified against the code as written.
trajectory_hybrid.py:225 returns `Failure(msg, is_recoverable=True)` for the missing-API-key
path with a comment at line 223 stating \"mirrors gemini\". yolo_hybrid.py:98 does the same,
with a docstring at line 72 saying \"mirrors the gemini/trajectory segmenters\". But
gemini.py:116 returns `Failure(msg)` (default is_recoverable=False) for the identical missing-
key condition. So the \"mirrors gemini\" comments are factually contradicted by gemini's own
code, and the same failure class is classified inconsistently across the three segmenters. The
finding's claim of zero backend consumers is also confirmed: a grep for `.is_recoverable` across
backend/ returns no matches (only field assignments exist; the only reads are in tests/), so no
caller branches on it today and there is no runtime behavior difference. This is a latent
convention/documentation inconsistency, not a runtime bug.\n\nOne nuance that does not refute
the finding but qualifies its framing: results.py:47 defines the field with the canonical
comment `is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash`,
which explicitly documents missing-API-key as recoverable. By that documented convention,
trajectory_hybrid/yolo_hybrid are correct and gemini.py:116 is the outlier ? the opposite of
what the \"mirrors gemini\" comments imply. Either way, the inconsistency and the misleading
comment are real, exactly as the finding states.\n\nSeverity low is accurate: no runtime impact
(no consumer), but a genuine, concrete stale/misleading comment (\"mirrors gemini\" when gemini
does the opposite) plus a cross-file inconsistency that a future maintainer adding retry logic
keyed on is_recoverable would trip over. This is a real convention-lens defect, not out-of-scope
speculation, so it is not REFUTED; it is correctly scoped as low rather than higher because
there is zero present-day behavioral divergence."
  },
  {
    "dimension": "convention-tests",
    "severity": "low",
    "verdict": "CONFIRMED",
    "file": "tests/test_tracknet_hybrid.py",
    "line": 100,
    "summary": "Of the F1/F2 pre-detection paths converted from bare []/None to Failure, only
the missing-API-key path is locked by a test; unknown-sport, unknown-provider, invalid/<=0
```

duration, and native-not-supported are all silently uncovered in both the tracknet and yolo suites.",
    "detail": "F1 converted five trajectory_hybrid pre-detection paths to Failure: unknown sport (KeyError -> trajectory_hybrid.py:198), missing key (line 225), unknown provider (KeyError -> line ~231), invalid/<=0 duration (line ~236), and _resolve_runner NotImplementedError / native-not-supported (line ~262). F2 converted the yolo equivalents in _resolve_provider (yolo_hybrid.py:80/98/106) and process_video invalid-duration (line 514). The tests only lock the missing-API-key conversion: test_tracknet_hybrid.py:100 (test_pipeline_no_api_key_is_failure) and test_yolo_hybrid.py:201 (test_yolo_hybrid_no_api_key). Grepping both test files for invalid-duration (get_video_duration return 0/negative), unknown/bad provider, unknown/bad sport, and NotImplementedError/native-runner yields no matches ? none of those four converted paths is asserted to return a Failure anywhere. So a regression that reverts any of them to `return []` (trajectory) or `return None` (yolo _resolve_provider) would reintroduce exactly the wf294 silent-'success' class and still pass the full suite. The healthcheck and no-trajectory paths (the earlier-reviewed change) ARE covered (test_tracknet_hybrid.py:129/152), and the legit-zero-windows empty and F3 all-errored/partial cases are covered ? it is specifically the F1/F2 config/env/input aborts other than missing-key that are unlocked.",
    "failure_scenario": "Someone edits _resolve_provider/_resolve_runner or the duration guard and accidentally restores `return []` / `return None` on the invalid-duration or native-not-supported path. The worker then writes status='success' with 0 segments for a broken video / unavailable detector (the wf294 silent failure), and the entire test suite stays green because no test exercises that path.",
    "suggested_fix": "Add parametrized tracknet twin tests and a yolo test asserting isinstance(res, Failure) for: get_video_duration <= 0 (invalid duration), a bad sport name (sport_registry KeyError), an unknown provider (provider_registry KeyError), and detector_impl='native' on tracknet (_resolve_runner NotImplementedError). Assert the message substring and, for consistency with the chosen convention, is_recoverable.",
    "verifier_reasoning": "Verified against the actual diff and test files. The diff (trajectory_hybrid.py / yolo_hybrid.py) converts five/four pre-detection early-returns from bare []/None to Failure: unknown sport (KeyError), missing API key, unknown provider (KeyError), invalid/<=0 duration, and _resolve_runner NotImplementedError (native-not-supported). Of these, only the missing-API-key conversion is locked by a test asserting isinstance(res, Failure): test_tracknet_hybrid.py:95 (test_pipeline_no_api_key_is_failure) and test_yolo_hybrid.py:210 (test_yolo_hybrid_no_api_key). The other tracknet/yolo Failure-asserting tests cover only healthcheck (tracknet:116), no-trajectory (tracknet:141), all-errored handoff F3 (equivalence:113, yolo:112), and legit-zero-windows-empty (tracknet:163) ? none of the four remaining converted config/input/detector paths.\\n\\nI specifically checked for false-negatives: (1) test_native_wasb_runner.py:502 (test_resolve_runner_native_refused_for_tracknet) asserts only that _resolve_runner RAISES NotImplementedError by calling _resolve_runner directly ? it does NOT call process_video and does NOT assert the trajectory_hybrid.py Failure conversion. (2) test_worker.py:857 exercises the `train` command's _resolve_train_detector path (rc 2), not process_video inference. (3) A cross-suite grep for 'Invalid video duration', invalid/unreadable duration, and unknown/bad sport/provider yields no test asserting a Failure from process_video for those paths. So a regression reverting any of the four unlocked paths to `return []`/`return None` would reintroduce the wf294 silent-'success' class and still pass the full suite.\\n\\nSeverity low is correct: the shipped code changes themselves are correct; this is a test-coverage/convention gap (some converted safety paths are not locked by tests), not a defect in production code."
  }
]

2. assistant (2026-07-06T18:06:14.934Z)

I'll review the surviving findings, verify them against the actual code, dedupe, rank, and provide an overall verdict. Let me start by reading the diff and the key code locations.

3. user (2026-07-06T18:06:17.816Z)

```
1       diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
2       index 503f6d8..7987278 100644
3       --- a/backend/pipeline/segmenters/trajectory_hybrid.py
4       +++ b/backend/pipeline/segmenters/trajectory_hybrid.py
5       @@ -22,7 +22,7 @@ import concurrent.futures
```

```
 import logging
 import os
 import time
-from typing import Any, Dict, List, Optional, Tuple
+from typing import Any, Dict, List, Optional, Tuple, Union

 import cv2

@@ -30,6 +30,9 @@ from backend.config.models import IndexingConfig
 from backend.pipeline.detectors.base import DetectorRunner
 from backend.pipeline.segmenters.base import BasePlaySegmenter
 from backend.providers import provider_registry
+
+# Standardized Success|Failure contract: a detector Failure is NOT a 0-rally result.
+from backend.results import Failure
 from backend.sports import sport_registry
 from backend.utils.paths import output_base
 from backend.utils.video import extract_video_chunk, get_video_duration
@@ -174,10 +177,14 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
         video_path: str,
         player_pool: Optional[List[str]] = None,
         max_frames: Optional[int] = None,
-    ) -> List[Dict[str, Any]]:
-        # override-ignore: the ABC declares the Result contract (Success|Failure)
-        # but every live segmenter still returns a bare list ? the documented
-        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
+    ) -> "Union[List[Dict[str, Any]], Failure]":
+        # Result contract (the ABC declares Success|Failure). Anything that means the run could
+        # NOT actually complete ? an unrunnable config (unknown sport/provider, missing API key),
+        # an unreadable video, an unavailable detector (native-not-supported), the env healthcheck
+        # not ready, or run_predict timing out / aborting with no trajectory ? returns a ``Failure``
+        # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match (a trajectory
+        # was produced but no candidate/AI-confirmed rallies) still returns a bare ``[]``. Every
+        # caller (worker.cmd_infer, main.py, orchestration) branches on isinstance(result, Failure).
         label = self.display_label
         # --- Sport profile + AI provider wiring (identical across segmenters) ---
         sport_name = self.config.sport
@@ -185,7 +192,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
             sport_profile = sport_registry.get(sport_name)
         except KeyError as e:
             logger.error(str(e))
-            return []
+            # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
+            return Failure(str(e))

         idx_cfg = self.config.indexing
         skip_ai = bool(idx_cfg.skip_ai_handoff)
@@ -205,24 +213,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
             if not api_key:
                 from backend.pipeline.segmenters._keyhint import api_key_hint

-                logger.error(
+                msg = (
                     f"{api_key_env_var} environment variable is not set! "
                     f"To run the {provider_name} handoff, set your API key. "
                     + api_key_hint(api_key_env_var)
                 )
-                return []
```

```
64      +                logger.error(msg)
65      +                # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
the run is
66      +                # marked failed instead of a silent empty "success" (mirrors gemini;
the worker's
67      +                # own 0-segment diagnostic names this missing-key case as a top cause).
68      +                return Failure(msg, is_recoverable=True)
69                   try:
70                       provider = provider_registry.get(
71                           provider_name, api_key=api_key, model_name=model_name
72                       )
73                   except KeyError as e:
74                       logger.error(str(e))
75      -                return []
76      +                return Failure(str(e))
77
78               video_duration = get_video_duration(video_path)
79               if video_duration <= 0:
80                   logger.error("Invalid video duration.")
81      -            return []
82      +            # An unreadable/broken video, not a 0-rally match.
83      +            return Failure(
84      +                message=f"{label}: invalid/unreadable video duration
({video_duration}).",
85      +            )
86               fps, frame_width = self._probe_fps_width(video_path)
87
88               # --- Runner config + windowing thresholds ---
89      @@ -233,9 +248,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
90                   runner, runner_params = self._resolve_runner(idx_cfg)
91               except NotImplementedError as e:
92                   # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
93      -            # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
94      +            # fusion with a tracknet substrate). An unavailable detector is a failure,
not a
95      +            # 0-rally match ? refuse cleanly as a Failure instead of crashing or a
false success.
96                   logger.error("%s: %s", label, e)
97      -            return []
98      +            return Failure(message=f"{label}: {e}")
99
100              velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
101              merge_gap = idx_cfg.dead_time_merge_gap
102     @@ -267,7 +283,12 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
103              ok, msg = runner.healthcheck()
104              if not ok:
105                  logger.error("%s detector environment not ready: %s", label, msg)
106     -            return []
107     +            # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
108     +            # failed/retryable instead of writing an empty-but-"successful" result.
109     +            return Failure(
110     +                message=f"{label} detector environment not ready: {msg}",
111     +                is_recoverable=True,
112     +            )
113              logger.info("%s detector env OK: %s", label, msg)
114
115              # --- Phase 2: trajectory -> candidate rally windows ---
116     @@ -278,7 +299,17 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
117              csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
118              if not csv_path:
119                  logger.error("%s produced no trajectory; aborting.", label)
120     -            return []
```

```
121     +            # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
122     +            # timeout_sec, or the detector env yielded nothing). This is a DETECTOR
failure ? NOT
123     +            # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
124     +            # rather than a silent 0-segment "success" (the wf294 incident).
125     +            return Failure(
126     +                message=(
127     +                    f"{label} detector produced no trajectory (timed out or aborted) ?
"
128     +                    "detector failure, not a 0-rally video."
129     +                ),
130     +                is_recoverable=True,
131     +            )
132
133              points = runner.parse_trajectory_csv(csv_path)
134              logger.info(
135     @@ -387,7 +418,11 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
136                      len(handoff_indices),
137                  )
138
139     -            def process_candidate_window(wi: int, window: Dict[str, Any]) ->
List[dict]:
140     +            def process_candidate_window(
141     +                wi: int, window: Dict[str, Any]
142     +            ) -> Tuple[List[dict], bool]:
143     +                """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
144     +                verdict (clip extraction failed, or the provider returned
metrics['error'])."""
145                      w_start = max(0, window["start"] - handoff_padding)
146                      w_end = min(video_duration, window["end"] + handoff_padding)
147                      w_dur = w_end - w_start
148     @@ -395,7 +430,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
149                      if not extract_video_chunk(
150                          video_path, w_start, w_dur, w_path, reencode=True
151                      ):
152     -                    return []
153     +                    return [], True  # extraction failed ? no verdict for this window
154                      prompt = sport_profile.get_prompt(chunk_duration=w_dur)
155                      segments, metrics = provider.analyze_video(
156                          w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
157     @@ -403,7 +438,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
158                      # B2: a provider failure returns [] with metrics["error"] set ? that is
159                      # "we don't know", NOT "no rallies in this window". Surface it loudly
so a
160                      # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
161     -                if metrics.get("error"):
162     +                errored = bool(metrics.get("error"))
163     +                if errored:
164                          logger.warning(
165                              "[Handoff %d] provider error ? window %d skipped: %s",
166                              wi + 1,
167     @@ -427,7 +463,7 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
168                          os.remove(w_path)
169                      except OSError:
170                          pass
171     -                return valid
172     +                return valid, errored
173
174                  with concurrent.futures.ThreadPoolExecutor(
175                      max_workers=ai_max_workers
176     @@ -438,8 +474,10 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
```

```
                        )
                        for wi in handoff_indices
                    ]
    -               for future in futures:
    -                   ai_segments.extend(future.result())
    +               window_results = [future.result() for future in futures]
    +           for valid, _errored in window_results:
    +               ai_segments.extend(valid)
    +           handoff_failed = sum(1 for _valid, errored in window_results if errored)
                try:
                    os.rmdir(handoff_dir)
                except OSError:
    @@ -448,6 +486,18 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
                    "Phase 3 completed. AI confirmed %d semantic play segments.",
                    len(ai_segments),
                )
    +           # wf294 (all-errored handoff): if EVERY handed-off window failed to yield a
verdict
    +           # (provider error / clip extraction failed), the run has zero information ?
a FAILURE,
    +           # not a 0-rally match. A single window that RAN and returned no rallies is
a legit empty
    +           # and keeps this from firing. (skip_ai has no provider, so it never reaches
here.)
    +           if handoff_indices and handoff_failed == len(handoff_indices):
    +               msg = (
    +                   f"{label} AI handoff produced no verdict: all {handoff_failed}
candidate "
    +                   "window(s) errored (provider error / clip extraction failed) ?
incomplete "
    +                   "run, not a 0-rally match."
    +               )
    +               logger.error(msg)
    +               return Failure(message=msg, is_recoverable=True)

            # --- Phase 4: DB population + candidate linking ---
            segments_data = []
    diff --git a/backend/pipeline/segmenters/yolo_hybrid.py
b/backend/pipeline/segmenters/yolo_hybrid.py
    index d9334f1..2e2e1fe 100644
    --- a/backend/pipeline/segmenters/yolo_hybrid.py
    +++ b/backend/pipeline/segmenters/yolo_hybrid.py
    @@ -22,11 +22,14 @@ import concurrent.futures
     import logging
     import os
     import time
    -from typing import Any, Dict, List, Tuple
    +from typing import Any, Dict, List, Tuple, Union

     from backend.pipeline.detectors.person_detector import PersonDetector
     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
     from backend.providers import provider_registry
    +
    +# Standardized Success|Failure contract: an unrunnable config is NOT a 0-rally result.
    +from backend.results import Failure
     from backend.sports import sport_registry
     from backend.utils.paths import output_base
     from backend.utils.video import extract_video_chunk, get_video_duration
    @@ -63,9 +66,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
         def _resolve_provider(self, video_path: str):
             """Phase 0: resolve sport profile + AI provider (with API key).

    -        Returns the configured provider instance, or ``None`` to signal that
    -        ``process_video`` should early-return ``[]`` (bad sport, missing API key,
    -        or unknown provider). Behaviour is identical to the inlined version.
```

```
235     +        Returns the configured provider instance, or a ``Failure`` (bad sport, missing
API key,
236     +        or unknown provider) that ``process_video`` propagates ? so an unrunnable
config is marked
237     +        FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
238     +        mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
239              """
240              # Get sport profile config
241              sport_name = self.config.sport
242     @@ -73,7 +77,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
243                  sport_registry.get(sport_name)
244              except KeyError as e:
245                  logger.error(str(e))
246     -            return None
247     +            return Failure(str(e))
248
249              # Get AI provider and model config
250              provider_name = self.config.ai_provider
251     @@ -85,12 +89,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
252              if not api_key:
253                  from backend.pipeline.segmenters._keyhint import api_key_hint
254
255     -            logger.error(
256     +            msg = (
257                      f"{api_key_env_var} environment variable is not set! "
258                      f"To run the {provider_name} segmenter, set your API key. "
259                      + api_key_hint(api_key_env_var)
260                  )
261     -            return None
262     +            logger.error(msg)
263     +            return Failure(msg, is_recoverable=True)
264
265              try:
266                  provider = provider_registry.get(
267     @@ -98,7 +103,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
268                  )
269              except KeyError as e:
270                  logger.error(str(e))
271     -            return None
272     +            return Failure(str(e))
273
274              logger.info(
275                  f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
276     @@ -320,9 +325,13 @@ class HybridYoloSegmenter(BasePlaySegmenter):
277              chunk_duration: float,
278              handoff_padding: float,
279              ai_max_workers: int,
280     -    ) -> List[dict]:
281     +    ) -> "Union[List[dict], Failure]":
282              """Phase 3: AI Provider Handoff ? re-encode each padded candidate window and
ask
283     -        the provider for semantic boundaries, in parallel. Verbatim move."""
284     +        the provider for semantic boundaries, in parallel.
285     +
286     +        Returns the confirmed segments, or a ``Failure`` when EVERY candidate window
failed to
287     +        yield a verdict (provider error / clip extraction failed) ? an incomplete run
has zero
288     +        information and must NOT be recorded as a 0-rally "success" (the wf294
class)."""
289              gemini_segments = []
290              gemini_dir = os.path.join(
```

```
                output_base(self.config), f"chunks_{provider_name}_handoff"
    @@ -333,7 +342,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows..."
            )

    -       def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
    +       def process_candidate_window(
    +           wi: int, window: Dict[str, Any]
    +       ) -> Tuple[List[dict], bool]:
    +           """Returns (valid_segments, errored) ? errored=True when this window
yielded NO verdict
    +           (clip extraction failed, or the provider returned metrics['error'])."""
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
    @@ -344,7 +357,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                    video_path, w_start, w_dur, w_path, reencode=True
                )
                if not success:
    -               return []
    +               return [], True  # extraction failed ? no verdict for this window

                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, metrics = provider.analyze_video(
    @@ -353,7 +366,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                # B2: a provider failure returns [] with metrics["error"] set ? that is
                # "we don't know", NOT "no rallies in this window". Surface it loudly so a
                # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
    -           if metrics.get("error"):
    +           errored = bool(metrics.get("error"))
    +           if errored:
                    logger.warning(
                        "[Handoff %d] provider error ? window %d skipped: %s",
                        wi + 1,
    @@ -380,7 +394,7 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                except OSError:
                    pass

    -           return valid_segments
    +           return valid_segments, errored

            with concurrent.futures.ThreadPoolExecutor(
                max_workers=ai_max_workers
    @@ -390,8 +404,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
                    for wi, w in enumerate(all_candidate_windows)
                ]
                # Iterate in submission order to preserve temporal ordering
    -           for future in futures:
    -               gemini_segments.extend(future.result())
    +           window_results = [future.result() for future in futures]
    +       for valid, _errored in window_results:
    +           gemini_segments.extend(valid)
    +       handoff_failed = sum(1 for _valid, errored in window_results if errored)

            try:
                os.rmdir(gemini_dir)
    @@ -401,6 +417,17 @@ class HybridYoloSegmenter(BasePlaySegmenter):
            logger.info(
                f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic play
segments."
            )
    +       # wf294 (all-errored handoff): if EVERY candidate window failed to yield a
verdict
```

```
351     +        # (provider error / clip extraction failed), the run has zero information ? a
FAILURE, not
352     +        # a 0-rally match. A window that RAN and returned no rallies keeps this from
firing.
353     +        if all_candidate_windows and handoff_failed == len(all_candidate_windows):
354     +            msg = (
355     +                f"yolo_hybrid AI handoff produced no verdict: all {handoff_failed}
candidate "
356     +                "window(s) errored (provider error / clip extraction failed) ?
incomplete run, "
357     +                "not a 0-rally match."
358     +            )
359     +            logger.error(msg)
360     +            return Failure(message=msg, is_recoverable=True)
361              return gemini_segments
362
363          def _populate_db(
364     @@ -456,19 +483,23 @@ class HybridYoloSegmenter(BasePlaySegmenter):
365
366              return segments_data
367
368     -    def process_video(
369     +    def process_video(  # type: ignore[override]
370              self,
371              video_id: str,
372              video_path: str,
373              player_pool: List[str] = None,
374              max_frames: int = None,
375     -    ) -> List[Dict[str, Any]]:
376     +    ) -> "Union[List[Dict[str, Any]], Failure]":
377     +        # Result contract: an unrunnable config (bad sport/provider, missing key) or an
unreadable
378     +        # video returns a Failure so the run is marked failed ? NOT a silent 0-rally
"success"
379     +        # (the wf294 class). A genuine empty match still returns a bare []. Every
caller branches
380     +        # on isinstance(result, Failure).
381              self.person.lazy_load_model()
382
383     -        # Phase 0: resolve sport profile + AI provider (early-returns [] on any
failure).
384     +        # Phase 0: resolve sport profile + AI provider (returns a Failure on any config
failure).
385              provider = self._resolve_provider(video_path)
386     -        if provider is None:
387     -            return []
388     +        if isinstance(provider, Failure):
389     +            return provider
390
391              # Re-read the config values needed downstream (pure deterministic reads).
392              idx_cfg = self.config.indexing
393     @@ -479,7 +510,10 @@ class HybridYoloSegmenter(BasePlaySegmenter):
394              video_duration = get_video_duration(video_path)
395              if video_duration <= 0:
396                  logger.error("Invalid video duration.")
397     -            return []
398     +            # An unreadable/broken video, not a 0-rally match.
399     +            return Failure(
400     +                message=f"yolo_hybrid: invalid/unreadable video duration
({video_duration})."
401     +            )
402
403              chunk_duration = idx_cfg.chunk_duration
404              chunk_overlap = idx_cfg.chunk_overlap
405     @@ -537,8 +571,8 @@ class HybridYoloSegmenter(BasePlaySegmenter):
```

```
406              except Exception:
407                  pass
408
409      -       # Phase 3: AI Provider Handoff
410      -       gemini_segments = self._run_ai_handoff(
411      +       # Phase 3: AI Provider Handoff (returns a Failure when every window errored ?
wf294).
412      +       handoff = self._run_ai_handoff(
413                  video_id,
414                  video_path,
415                  video_duration,
416      @@ -551,9 +585,11 @@ class HybridYoloSegmenter(BasePlaySegmenter):
417                  handoff_padding,
418                  ai_max_workers,
419              )
420      +       if isinstance(handoff, Failure):
421      +           return handoff
422
423              # Phase 4: DB Population
424      -       return self._populate_db(video_id, gemini_segments, model_name)
425      +       return self._populate_db(video_id, handoff, model_name)
426
427
428       # Register the segmenter
429      diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
430      index 461d7ca..88c50f6 100644
431      --- a/backend/worker/__main__.py
432      +++ b/backend/worker/__main__.py
433      @@ -114,6 +114,42 @@ def _write_report_json(
434               )
435
436
437      +def _serialize_and_push_outputs(
438      +    scratch: str,
439      +    out_uri: str,
440      +    video_id: str,
441      +    segments: List[dict],
442      +    status: str,
443      +    storage_cfg: dict,
444      +    video_uri: str = "",
445      +) -> List[str]:
446      +    """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
447      +    ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
448      +
449      +    Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
450      +    ``status="failed"`` report.json in the bucket ? the control plane's source of truth
451      +    (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
452      +    report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
453      +    EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``."""
454      +    out_local = os.path.join(scratch, "outputs", video_id)
455      +    os.makedirs(out_local, exist_ok=True)
456      +    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
457      +    _write_report_json(
458      +        video_id,
459      +        segments,
460      +        status,
461      +        os.path.join(out_local, "report.json"),
462      +        video_uri=video_uri,
463      +    )
464      +    out_prefix = out_uri.rstrip("/")
465      +    pushed: List[str] = []
```

```
    +    for fn in sorted(os.listdir(out_local)):
    +        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
    +        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
    +        pushed.append(uri)
    +    return pushed
    +
    +
     def _run_startup_checks(config: Any) -> bool:
         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
    @@ -237,9 +273,26 @@ def cmd_infer(args: argparse.Namespace) -> int:
         )

         def _fail(rc: int) -> int:
    -        # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments)
so a remote
    -        # dispatcher's bucket-poll terminates FAST + accurately instead of hanging
until its poll
    -        # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
    +        # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
    +        # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
    +        # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
    +        # truth (submit-cache probe, restart-recovery) and the alpha-user result
surface see an
    +        # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
    +        # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
    +        # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
    +        try:
    +            _serialize_and_push_outputs(
    +                scratch,
    +                args.out_uri,
    +                video_id,
    +                [],
    +                "failed",
    +                storage_cfg,
    +                str(getattr(args, "video_uri", "") or ""),
    +            )
    +        except Exception as e:  # never mask the real failure on an artifact-push
hiccup
    +            logger.warning("[worker] could not push failure artifacts: %s", e)
    +        # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
             if getattr(args, "done_uri", None):
    +            _write_done_marker(
    +                args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
    @@ -313,24 +366,16 @@ def cmd_infer(args: argparse.Namespace) -> int:
             )

         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
    -    out_local = os.path.join(scratch, "outputs", video_id)
    -    os.makedirs(out_local, exist_ok=True)
    -    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    -    _write_report_json(
    +    pushed = _serialize_and_push_outputs(
    +        scratch,
    +        args.out_uri,
             video_id,
```

```
518              segments,
519              status,
520      -       os.path.join(out_local, "report.json"),
521      -       video_uri=str(getattr(args, "video_uri", "") or ""),
522      +       storage_cfg,
523      +       str(getattr(args, "video_uri", "") or ""),
524          )
525
526      -    out_prefix = args.out_uri.rstrip("/")
527      -    pushed = []
528      -    for fn in sorted(os.listdir(out_local)):
529      -        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
530      -        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
531      -        pushed.append(uri)
532      -
533          print(
534              f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
535          )
536      diff --git a/tests/test_tracknet_hybrid.py b/tests/test_tracknet_hybrid.py
537      index 64909e7..c157ed5 100644
538      --- a/tests/test_tracknet_hybrid.py
539      +++ b/tests/test_tracknet_hybrid.py
540      @@ -5,6 +5,7 @@ healthcheck gating, candidate persistence, AI handoff, and DB
population/linking
541       from unittest.mock import MagicMock, patch
542
543       from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
544      +from backend.results import Failure
545
546
547       def _window(start=1.0, end=4.0):
548      @@ -91,7 +92,7 @@ def test_pipeline_happy_path(
549
550
551       @_patched
552      -def test_pipeline_no_api_key_returns_empty(
553      +def test_pipeline_no_api_key_is_failure(
554           mock_runner_cls,
555           mock_probe,
556           mock_dur,
557      @@ -99,12 +100,16 @@ def test_pipeline_no_api_key_returns_empty(
558           mock_provider_registry,
559           mock_env,
560       ):
561      +    """A missing API key (with skip_ai_handoff=False) is an unrunnable misconfig ? a
Failure, NOT a
562      +    0-rally []. This is a wf294-class silent-failure the worker must mark failed, not
"success"."""
563           mock_env.return_value = None
564           mock_dur.return_value = 30.0
565           mock_runner_cls.return_value = _make_runner()
566
567           seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
568      -    assert seg.process_video("v1", "clip.mp4") == []
569      +    res = seg.process_video("v1", "clip.mp4")
570      +    assert isinstance(res, Failure) and res.is_recoverable
571      +    assert "API_KEY" in res.message
572
573
574       @_patched
575      @@ -124,7 +129,10 @@ def test_pipeline_healthcheck_failure_aborts(
576
577           seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
578           res = seg.process_video("v1", "clip.mp4")
```

```
-    assert res == []
+    # A detector-env failure is a Failure, NOT a 0-rally [] ? so the caller marks the
run failed
+    # instead of writing an empty "success".
+    assert isinstance(res, Failure)
+    assert "environment not ready" in res.message
         # Should bail before ever running inference.
         mock_runner_cls.return_value.run_predict.assert_not_called()

@@ -144,7 +152,36 @@ def test_pipeline_no_trajectory_aborts(
         mock_provider_registry.get.return_value = MagicMock()

         seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
-    assert seg.process_video("v1", "clip.mp4") == []
+    res = seg.process_video("v1", "clip.mp4")
+    # A detector timeout/abort (run_predict yielded no trajectory) is a Failure, NOT a
0-rally [] ?
+    # this is exactly the wf294 silent-failure that the caller must now mark
failed/retryable.
+    assert isinstance(res, Failure)
+    assert "no trajectory" in res.message and res.is_recoverable
+
+
+@_patched
+def test_pipeline_zero_windows_is_legit_empty_not_failure(
+    mock_runner_cls,
+    mock_probe,
+    mock_dur,
+    mock_extract,
+    mock_provider_registry,
+    mock_env,
+):
+    """The other side of the wf294 distinction: the detector RAN fine (a trajectory was
produced +
+    parsed) but yielded no candidate rally windows. That is a legitimately empty match
? a bare []
+    (0-rally "success"), NOT a Failure."""
+    mock_env.return_value = "dummy_key"
+    mock_dur.return_value = 30.0
+    mock_extract.return_value = True
+    mock_runner_cls.return_value = _make_runner(windows=[])  # trajectory ok, zero
windows
+    mock_provider_registry.get.return_value = MagicMock()
+
+    seg = TrackNetHybridSegmenter(MagicMock(), {"indexing": {}})
+    res = seg.process_video("v1", "clip.mp4")
+    assert res == [] and not isinstance(res, Failure)
+    # The detector DID run (a genuine empty, not an abort).
+    mock_runner_cls.return_value.run_predict.assert_called_once()


 @_patched
diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
index 0080f4d..0270e41 100644
--- a/tests/test_trajectory_hybrid_equivalence.py
+++ b/tests/test_trajectory_hybrid_equivalence.py
@@ -14,6 +14,7 @@ import pytest
 from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
 from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.results import Failure

 TWINS = [
     (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
```

```
637     @@ -111,8 +112,10 @@ def test_provider_offsets_applied_identically(cls, family,
detector):
638
639      @pytest.mark.parametrize("cls,family,detector", TWINS)
640      def test_provider_failure_is_surfaced_not_silent(cls, family, detector, caplog):
641     -    """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
642     -    treated as 'no rallies in this window'. The run still returns [] (no analysis
done)."""
643     +    """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
644     +    silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
645     +    zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
646     +    match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
647          import logging
648
649          provider = MagicMock()
650     @@ -139,13 +142,45 @@ def test_provider_failure_is_surfaced_not_silent(cls, family,
detector, caplog):
651                  logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
652              ):
653                  res = seg.process_video("v1", "clip.mp4")
654     -    assert res == []  # no analysis for the failed window
655     +    assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored
-> no verdict
656          assert any(
657              "provider error" in r.message and "generate failed" in r.message
658              for r in caplog.records
659          ), [r.message for r in caplog.records]
660
661
662     +@pytest.mark.parametrize("cls,family,detector", TWINS)
663     +def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
664     +    """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
665     +    no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
666     +    handed off; the first returns a rally, the second errors -> a Success with the
first rally."""
667     +    provider = MagicMock()
668     +    provider.analyze_video.side_effect = [
669     +        ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
670     +        ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
671     +    ]
672     +    db = MagicMock()
673     +    db.add_candidate_segment.side_effect = [55, 56]
674     +    db.add_play_segment.return_value = 200
675     +    two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
676     +    with patch.object(
677     +        cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path":
"w"})
678     +    ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
679     +        "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
680     +        return_value=30.0,
681     +    ), patch(
682     +        "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
683     +        return_value=True,
684     +    ), patch(
685     +        "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
686     +    ) as preg, patch("os.environ.get", return_value="dummy_key"):
687     +        preg.get.return_value = provider
688     +        seg = cls(db, {"indexing": {}})
```

```
+        res = seg.process_video("v1", "clip.mp4")
+    assert not isinstance(res, Failure)
+    assert len(res) == 1 and res[0]["start_time"] == 0.5
+
+
 def test_base_is_not_registered():
     from backend.pipeline.segmenters.base import segmenter_registry

diff --git a/tests/test_worker.py b/tests/test_worker.py
index 96edbee..8fb5e16 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -10,6 +10,7 @@ import json
 import pytest

 from backend.config import load_config
+from backend.results import Failure
 from backend.worker import __main__ as wmain

 # ---------------------------------------------------------------- pure helpers
@@ -191,7 +192,7 @@ def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):


 class _EmptySeg:
-    """A segmenter that finds nothing ? the cold-start failure mode."""
+    """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""

     def __init__(self, db, config):
         pass
@@ -200,6 +201,20 @@ class _EmptySeg:
         return []


+class _FailSeg:
+    """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB
produced no
+    trajectory), which is NOT the same as a 0-rally match. This is the wf294 shape."""
+
+    def __init__(self, db, config):
+        pass
+
+    def process_video(self, video_id, path, max_frames=None):
+        return Failure(
+            message="WASB detector produced no trajectory (timed out or aborted)",
+            is_recoverable=True,
+        )
+
+
 def test_setup_logging_levels():
     import logging

@@ -246,12 +261,112 @@ def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog)
         assert "0 segments" in msgs and "vidZ" in msgs
         assert "detector_impl=native" in msgs  # surfaces the resolved detector
         assert "native runner UNAVAILABLE" in msgs  # points at the likely cause
-    assert (
-        json.loads((out / "outputs" / "vidZ" / "report.json").read_text())[
-            "segment_count"
+    # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
+    # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as
status="failed").
+    report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
+    assert report["segment_count"] == 0 and report["status"] == "success"
```

```
750     +
751     +
752     +def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
753     +    tmp_path, monkeypatch
754     +):
755     +    """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
756     +    as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
757     +    status="failed" so the control plane / alpha-user surface can show an error /
retry."""
758     +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
759     +    monkeypatch.setattr(
760     +        wmain.validator_registry,
761     +        "run_all_with_context",
762     +        lambda path, cfg: ([_OKResult()], {}),
763     +    )
764     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
765     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
766     +    video = tmp_path / "in.mp4"
767     +    video.write_bytes(b"FAKE")
768     +    out = tmp_path / "bucket"
769     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
770     +
771     +    rc = wmain.main(
772     +        [
773     +            "infer",
774     +            "--video-uri",
775     +            str(video),
776     +            "--out-uri",
777     +            str(out),
778     +            "--scratch",
779     +            str(tmp_path / "s"),
780     +            "--segmenter",
781     +            "wasb_hybrid",
782     +            "--done-uri",
783     +            str(done),
784     +        ]
785     +    )
786     +    # A detector failure is rc 3 (distinct from validation's rc 2 and a clean rc 0).
787     +    assert rc == 3
788     +    # report.json is written as an explicit FAILURE, not a false empty "success".
789     +    report = json.loads((out / "outputs" / "vidX" / "report.json").read_text())
790     +    assert report["status"] == "failed" and report["segment_count"] == 0
791     +    assert report["segments"] == []
792     +    # ...and the completion marker the cloud dispatcher bucket-polls carries the same
verdict.
793     +    marker = json.loads(done.read_text())
794     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
795     +    assert marker["segment_count"] == 0 and marker["video_id"] == "vidX"
796     +
797     +
798     +def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path,
monkeypatch):
799     +    """End-to-end: the ACTUAL WasbHybridSegmenter no-trajectory abort (the wf294 shape)
flows
800     +    through cmd_infer -> _fail(3) -> a status="failed" report.json + done-marker. Locks
the two
801     +    halves (segmenter Failure + worker report/marker) TOGETHER, not just via the
_FailSeg stub."""
802     +    from unittest.mock import MagicMock
803     +
804     +    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
805     +
806     +    runner = MagicMock()
```

```
807     +    runner.healthcheck.return_value = (True, "env ok")
808     +    runner.run_predict.return_value = None  # WASB timed out / aborted: no trajectory
produced
809     +
810     +    # Force the real base process_video down to its no-trajectory abort without a
GPU/video.
811     +    monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg:
(runner, {}))
812     +    monkeypatch.setattr(
813     +        WasbHybridSegmenter, "_probe_fps_width", lambda *a, **k: (30.0, 1280.0)
814     +    )
815     +    monkeypatch.setattr(
816     +        "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration", lambda p:
30.0
817     +    )
818     +    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
819     +    monkeypatch.setattr(
820     +        wmain.validator_registry, "run_all_with_context", lambda path, cfg:
([_OKResult()], {})
821     +    )
822     +    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
823     +    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
824     +
825     +    video = tmp_path / "in.mp4"
826     +    video.write_bytes(b"FAKE")
827     +    out = tmp_path / "bucket"
828     +    done = tmp_path / "bucket" / "_dispatch" / "tok.done"
829     +
830     +    rc = wmain.main(
831     +        [
832     +            "infer",
833     +            "--video-uri",
834     +            str(video),
835     +            "--out-uri",
836     +            str(out),
837     +            "--scratch",
838     +            str(tmp_path / "s"),
839     +            "--segmenter",
840     +            "wasb_hybrid",
841     +            "--done-uri",
842     +            str(done),
843     +            "--set",
844     +            "indexing.skip_ai_handoff=true",  # no API key needed; isolate the detector
abort
845             ]
846     -       == 0
847         )
848     +    assert rc == 3
849     +    report = json.loads((out / "outputs" / "vidR" / "report.json").read_text())
850     +    assert report["status"] == "failed" and report["segment_count"] == 0
851     +    marker = json.loads(done.read_text())
852     +    assert marker["returncode"] == 3 and marker["status"] == "failed"
853
854
855      def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
856     @@ -266,20 +381,30 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path,
monkeypatch):
857         video.write_bytes(b"X")
858         cfg = tmp_path / "config.json"
859         cfg.write_text("{}")
860     +    out = tmp_path / "o"
861     +    done = tmp_path / "o" / "_dispatch" / "tok.done"
862         rc = wmain.main(
863             [
```

```
                "infer",
                "--video-uri",
                str(video),
                "--out-uri",
    -           str(tmp_path / "o"),
    +           str(out),
                "--config",
                str(cfg),
                "--scratch",
                str(tmp_path / "s"),
    +           "--done-uri",
    +           str(done),
            ]
        )
        assert rc == 2
    +   # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
    +   # success): report.json status="failed" + the done-marker rc 2 / status="failed".
    +   report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
    +   assert report["status"] == "failed" and report["segment_count"] == 0
    +   marker = json.loads(done.read_text())
    +   assert marker["returncode"] == 2 and marker["status"] == "failed"


     # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push
bundle ---
    diff --git a/tests/test_yolo_hybrid.py b/tests/test_yolo_hybrid.py
    index 38e78b2..09da6c2 100644
    --- a/tests/test_yolo_hybrid.py
    +++ b/tests/test_yolo_hybrid.py
    @@ -11,6 +11,7 @@ import os
     from unittest.mock import MagicMock, patch

     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
    +from backend.results import Failure


     def _make_cap_mock(num_frames=300):
    @@ -110,8 +111,10 @@ def test_yolo_hybrid_segmenter(
     def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
         mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap, caplog
     ):
    -    """B2/P17: a provider failure ([], metrics['error']) must be logged loudly, not
silently
    -    treated as 'no rallies'. The run still returns [] (no analysis done)."""
    +    """B2/P17 + F3 (wf294): a provider failure ([], metrics['error']) must be logged
loudly, not
    +    silently treated as 'no rallies'. When it errors on the ONLY candidate window the
whole run has
    +    zero verdict, so process_video now returns a Failure (was a bare [] that looked
like a 0-rally
    +    match) ? the worker marks it failed/retryable instead of a silent empty
'success'."""
         import logging

         mock_env_get.return_value = "dummy_api_key"
    @@ -147,13 +150,49 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
             logging.WARNING, logger="backend.pipeline.segmenters.yolo_hybrid"
         ):
             res = segmenter.process_video("v1", "dummy.mp4")
    -    assert res == []  # no analysis for the failed window
    +    assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored
-> no verdict
         assert any(
             "provider error" in r.message and "generate failed" in r.message
```

```
921              for r in caplog.records
922          ), [r.message for r in caplog.records]
923
924
925      +@patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
926      +@patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
927      +@patch("backend.pipeline.segmenters.yolo_hybrid.provider_registry")
928      +@patch("os.environ.get")
929      +def test_yolo_hybrid_partial_handoff_error_is_a_success_not_a_failure(
930      +    mock_env_get, mock_provider_registry, mock_extract, mock_get_dur
931      +):
932      +    """The boundary of F3: SOME windows errored but at least one RAN, so the run is a
Success (the
933      +    first window's rally), NOT a Failure. Stage 2 is patched to hand off two windows
directly."""
934      +    mock_env_get.return_value = "dummy_api_key"
935      +    mock_get_dur.return_value = 30.0
936      +    mock_extract.return_value = True
937      +
938      +    mock_provider = MagicMock()
939      +    mock_provider.analyze_video.side_effect = [
940      +        ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
941      +        ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
942      +    ]
943      +    mock_provider_registry.get.return_value = mock_provider
944      +
945      +    db_mock = MagicMock()
946      +    db_mock.add_play_segment.return_value = "yolo_segment_1"
947      +    two_windows = [{"start": 1.0, "end": 4.0}, {"start": 10.0, "end": 13.0}]
948      +
949      +    segmenter = HybridYoloSegmenter(db_mock, {"indexing": {"use_gpu": False}})
950      +    segmenter.person.model = MagicMock()  # short-circuit lazy model load
951      +    with patch.object(
952      +        HybridYoloSegmenter, "_collect_candidate_windows", return_value=two_windows
953      +    ), patch.object(
954      +        HybridYoloSegmenter, "_persist_candidates", return_value=[None, None]
955      +    ):
956      +        res = segmenter.process_video("v1", "dummy.mp4")
957      +    assert not isinstance(res, Failure)
958      +    assert len(res) == 1 and res[0]["start_time"] == 0.5
959      +
960      +
961       @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
962       @patch("backend.pipeline.segmenters.yolo_hybrid.get_video_duration")
963       @patch("backend.pipeline.segmenters.yolo_hybrid.extract_video_chunk")
964      @@ -162,7 +201,8 @@ def test_yolo_hybrid_provider_failure_is_surfaced_not_silent(
965       def test_yolo_hybrid_no_api_key(
966           mock_env_get, mock_provider_registry, mock_extract, mock_get_dur, mock_cap
967       ):
968      -    """Segmenter should return empty when GEMINI_API_KEY is missing."""
969      +    """A missing API key is an unrunnable misconfig ? a Failure, NOT a 0-rally [] that
the worker
970      +    would write as a silent status="success" (the wf294 class; yolo_hybrid is the
default segmenter)."""
971           mock_env_get.return_value = None
972
973           db_mock = MagicMock()
974      @@ -172,7 +212,8 @@ def test_yolo_hybrid_no_api_key(
975           segmenter.person.model = MagicMock()  # short-circuit lazy model load
976
977           res = segmenter.process_video("v1", "dummy.mp4")
978      -    assert res == []
979      +    assert isinstance(res, Failure) and res.is_recoverable
980      +    assert "API_KEY" in res.message
981
```

```
982
983      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
984
```

4. user (2026-07-06T18:06:18.616Z)

```
140             seg = cls(db, {"indexing": {}})
141             with caplog.at_level(
142                 logging.WARNING, logger="backend.pipeline.segmenters.trajectory_hybrid"
143             ):
144                 res = seg.process_video("v1", "clip.mp4")
145         assert isinstance(res, Failure) and res.is_recoverable  # all (1) windows errored ->
no verdict
146         assert any(
147             "provider error" in r.message and "generate failed" in r.message
148             for r in caplog.records
149         ), [r.message for r in caplog.records]
150
151
152     @pytest.mark.parametrize("cls,family,detector", TWINS)
153     def test_partial_handoff_error_is_a_success_not_a_failure(cls, family, detector):
154         """The boundary of F3: when SOME candidate windows error but at least one RAN (even
if it found
155         no rallies), the run is NOT a failure ? only an all-errored handoff is. Here two
windows are
156         handed off; the first returns a rally, the second errors -> a Success with the first
rally."""
157         provider = MagicMock()
158         provider.analyze_video.side_effect = [
159             ([{"start_time": 0.5, "end_time": 2.0}], {}),  # window 0: a real verdict
160             ([], {"error": "generate failed after retries: 503"}),  # window 1: errored
161         ]
162         db = MagicMock()
163         db.add_candidate_segment.side_effect = [55, 56]
164         db.add_play_segment.return_value = 200
165         two_windows = [_window(1.0, 4.0), _window(10.0, 13.0)]
166         with patch.object(
167             cls, "_build_runner", return_value=(_runner(two_windows), {"weights_path": "w"})
168         ), patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), patch(
169             "backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
170             return_value=30.0,
171         ), patch(
172             "backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
173             return_value=True,
174         ), patch(
175             "backend.pipeline.segmenters.trajectory_hybrid.provider_registry"
176         ) as preg, patch("os.environ.get", return_value="dummy_key"):
177             preg.get.return_value = provider
178             seg = cls(db, {"indexing": {}})
179             res = seg.process_video("v1", "clip.mp4")
180         assert not isinstance(res, Failure)
181         assert len(res) == 1 and res[0]["start_time"] == 0.5
182
183
184     def test_base_is_not_registered():
185         from backend.pipeline.segmenters.base import segmenter_registry
186
187         available = segmenter_registry.list_available()
188         assert "tracknet_hybrid" in available
189         assert "wasb_hybrid" in available
190         # The base engine itself must not appear as a selectable segmenter.
191         assert not any(
192             isinstance(cls, type) and cls is TrajectoryHybridSegmenter
193             for cls in available.values()
194         )
```

```
195
196
197     @pytest.mark.parametrize("cls,family,detector", TWINS)
198     def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
199         """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
identical:
```

5. assistant (2026-07-06T18:06:21.944Z)

Now let me verify the key claims. Let me check the config defaults and the results.py definition.

6. user (2026-07-06T18:06:23.949Z)

```
248         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
249         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
250         window_blob_factor: float = 4.0
251         log_yolo_candidates: bool = True
252         store_yolo_candidates: bool = True
253         ai_handoff_padding: float = 2.0
254         ai_max_workers: int = 5
255         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
256         # chunks of a high-bitrate (4K) source blow past the provider upload cap
257         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
258         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
259         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
```

7. user (2026-07-06T18:06:24.958Z)

```
40
41     @dataclass
42     class Failure:
43         """Represents a failure with a human-readable message and optional details."""
44
45         message: str
46         details: Optional[dict[str, Any]] = None
47         is_recoverable: bool = False  # e.g. missing API key (recoverable) vs internal crash
48
49         def is_success(self) -> bool:
50             return False
51
52         def unwrap(self) -> Any:
53             """Raise to make misuse obvious in code that assumes success."""
54             raise RuntimeError(f"Attempted to unwrap a Failure: {self.message}")
```

8. user (2026-07-06T18:06:25.836Z)

```
108             if not api_key:
109                 from backend.pipeline.segmenters._keyhint import api_key_hint
110
111                 msg = (
112                     f"{api_key_env_var} is not set ? the {provider_name} segmenter cannot
run. "
113                     f"Set your API key. " + api_key_hint(api_key_env_var)
114                 )
115                 logger.error(msg)
116                 return Failure(msg)
117
118             try:
119                 provider = provider_registry.get(
120                     provider_name, api_key=api_key, model_name=model_name
121                 )
```

9. assistant (2026-07-06T18:06:35.263Z)

All three findings' load-bearing facts confirmed:

1. `ai_max_workers` default = 5, `ai_handoff_padding` default = 2.0 (models.py:253-254) ? confirms both race findings.
2. `results.py:47` documents missing-API-key as recoverable; `gemini.py:116` returns `Failure(msg)` (default `is_recoverable=False`) ? confirms the inconsistency between the "mirrors gemini" comment and gemini's actual code.

Now let me verify the two race findings more carefully ? specifically whether the offset actually maps to the window and whether concurrency is real. The critical question for the race findings: does `_window(1.0, 4.0)` vs `_window(10.0, 13.0)` actually produce distinct offsets, and is arrival-order binding real. Both verifiers already established this. Let me confirm the trajectory test's window ordering and the offset application line.

10. user (2026-07-06T18:06:36.448Z)

```
418                     len(handoff_indices),
419                 )
420
421                 def process_candidate_window(
422                     wi: int, window: Dict[str, Any]
423                 ) -> Tuple[List[dict], bool]:
424                     """Returns (valid_segments, errored) ? errored=True when this window
yielded NO
425                     verdict (clip extraction failed, or the provider returned
metrics['error'])."""
426                     w_start = max(0, window["start"] - handoff_padding)
427                     w_end = min(video_duration, window["end"] + handoff_padding)
428                     w_dur = w_end - w_start
429                     w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
430                     if not extract_video_chunk(
431                         video_path, w_start, w_dur, w_path, reencode=True
432                     ):
433                         return [], True  # extraction failed ? no verdict for this window
434                     prompt = sport_profile.get_prompt(chunk_duration=w_dur)
435                     segments, metrics = provider.analyze_video(
436                         w_path, prompt, chunk_duration=w_dur, label=f"Handoff {wi + 1}"
437                     )
438                     # B2: a provider failure returns [] with metrics["error"] set ? that is
439                     # "we don't know", NOT "no rallies in this window". Surface it loudly so
a
440                     # partial run isn't mistaken for a complete one (mirrors the gemini
segmenter).
441                     errored = bool(metrics.get("error"))
442                     if errored:
443                         logger.warning(
444                             "[Handoff %d] provider error ? window %d skipped: %s",
445                             wi + 1,
446                             wi,
447                             metrics["error"],
448                         )
449                     valid = []
450                     for seg in segments:
451                         try:
452                             seg["start_time"] = float(seg["start_time"]) + w_start
453                             seg["end_time"] = float(seg["end_time"]) + w_start
454                             seg["_candidate_id"] = candidate_ids[wi]
455                             valid.append(seg)
456                         except (ValueError, TypeError, KeyError) as e:
457                             logger.warning(
458                                 "Dropping segment with unparseable timestamps: %s ? %s",
459                                 seg,
460                                 e,
```

```
                            )
                    try:
                        os.remove(w_path)
                    except OSError:
                        pass
                    return valid, errored

                with concurrent.futures.ThreadPoolExecutor(
                    max_workers=ai_max_workers
                ) as executor:
                    futures = [
                        executor.submit(
                            process_candidate_window, wi, all_candidate_windows[wi]
                        )
                        for wi in handoff_indices
                    ]
                    window_results = [future.result() for future in futures]
                for valid, _errored in window_results:
                    ai_segments.extend(valid)
                handoff_failed = sum(1 for _valid, errored in window_results if errored)
                try:
                    os.rmdir(handoff_dir)
```

11. assistant (2026-07-06T18:06:48.772Z)

Confirmed. The critical race detail: line 452 applies `w_start` (the window-specific offset) to whatever `segments` the provider returned. But `provider.analyze_video` is a MagicMock with `side_effect=[rally, error]` consumed in **arrival order**, while both windows run concurrently in the ThreadPoolExecutor (max_workers=5). So the rally can be bound to window 1 (offset 8.0) instead of window 0 (offset 0.0), making `start_time == 8.5` instead of `0.5`. The `len(res)==1` and Success verdict hold regardless. This is a genuine flaky-test race in the newly added test.

Both race findings (trajectory + yolo) are confirmed and are the same defect class in two files. They are NOT the same test ? different files, different segmenters ? so they should stay as two findings, but ranked adjacently.

All four findings are confirmed. None are fine-on-reflection. None are production defects ? all are test/convention issues. Let me assemble the verdict.

The two race findings are the most severe (they will cause intermittent CI failures ? real broken-CI risk). The two convention/coverage findings are low (no runtime impact). None block the merge in a strict sense, but the flaky tests are worth fixing before merge because they will produce spurious red builds. Given the user's own pre-LGTM gate emphasizes a reliably-green suite, I'll mark the two flaky-test races as must-fix-before-merge, and the two low-severity gaps as not-must-fix.

12. user (2026-07-06T18:07:23.345Z)

Structured output provided successfully